

## Javascript

```

case 'IsNotValued':
  //The field is required if the other field is not valued.
  if (ELRMessage.getField(path_OtherField) == null && ELRMessage.getField(path_ReqField) ==
null) {
    errorMsg += ' is not valued.';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
  }
  break;
case 'IsSpecificValue':
  //The field is required if the other field equals a specific value.
  if (ELRMessage.getField(path_OtherField) != otherSpecificValue && ELRMessage.getField(path
_ReqField) == null) {
    errorMsg += ' equals "' + otherSpecificValue + '".';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
  }
  break;
case 'IsNotSpecificValue':
  //The field is required if the other field does not equal a specific value.
  if (ELRMessage.getField(path_OtherField) != otherSpecificValue && ELRMessage.getField(path
_ReqField) == null) {
    errorMsg += ' does not equal "' + otherSpecificValue + '".';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
  }
  break;
default:
  break;
}
}

//FUNCTION: fnRequiredIfTwoFieldsMissing
//Function to check if a field is present if it is required when two other fields are missing.
function fnRequiredIfTwoFieldsMissing (valDefinition, errorCategory, path_ReqField, pathHL7_Mis
singData1, pathHL7_MissingData2, msgErrorLoc_ReqField, msgErrorLoc_MissingData1, msgErrorLoc
_MissingData2, fieldNotation) {
  if ((ELRMessage.getField(pathHL7_MissingData1) == null) && (ELRMessage.getField(pathHL7_Mis
singData2) == null) && (ELRMessage.getField(path_ReqField) == null)) {
    var errorMsg = 'This message is missing ' + msgErrorLoc_ReqField + ' (' + path_ReqField
.substr(path_ReqField.lastIndexOf('/')+1) + ')' + ' which is required if '
    + msgErrorLoc_MissingData1 + ' (' + pathHL7_MissingData1.substr(pathHL7_MissingData1.last
IndexOf('/')+1) + ')'
    + ' and ' + msgErrorLoc_MissingData2 + ' (' + pathHL7_MissingData2.substr(pathHL7_Missing
Data2.lastIndexOf('/')+1) + ')' + ' are not valued.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, path_ReqField, errorMsg,
'CONDITIONAL PREDICATE', 'Missing data');
  }
}

//FUNCTION: fnValidateFormat
//Function to validate various formats.
function fnValidateFormat (valDefinition, errorCategory, fieldNotation, pathHL7, checkNull,
fieldReqLength, invalidPattern, validationFormat) {
  var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must ';
  var fieldValue = ELRMessage.getField(pathHL7);

```

## Javascript

```

if (fieldValue != null || validationFormat == 'AlphaOnly') {
  switch(validationFormat) {
    case 'ISO':
      if (fieldValue.match(/^([0-2])(\.[0-9]|\.([1-9][0-9]*)*)?$/)) == null) {
        errorMsg += 'follow the required ISO OID format. The value in this message "' + field
Value + '" does not follow the correct format.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - ISO');
      }
      break;
    case 'CLIA':
      if (fieldValue.match(/[0-9]{2}D[0-9]{7}/) == null) {
        errorMsg += 'be a valid CLIA identifier format. The value in this message "' + fieldValue
+ '" does not follow the correct format.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - CLIA');
      }
      break;
    case 'ZipCode':
      if ((fieldValue != null) && (fieldValue.match(/^\d{5}(-\d{4})?/) == null)) {
        errorMsg += 'be formatted as 99999[-9999] for US Zip or ZIP +4 codes. The value in this
message "' + fieldValue + '" does not follow the correct format.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - zip');
      }
      break;
    case 'LOINC':
      errorMsg += 'be a valid LOINC code identifier format. The value in this message "' + field
Value + '" does not follow the correct format.';
      if (fieldValue.match(/^([a-zA-Z0-9]{1,8}-\d{1})/) == null) {
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - LOINC');
      }
      break;
    case 'SSN':
      if (fieldValue.match(/(?!(\d)\1{8})(?!(\d)\2{2}(\d)\2{2}(\d)\2{3})(?:?!000|666|9|123\d\d)\d{3}
(?:00|45)\d\d(?:0000|6789)\d{4})/) == null) {
        errorMsg += 'be formatted as ##### (9 digits, no special characters) and cannot be all
the same number, cannot be 123-45-6789. The value in this message "' + fieldValue + '" does
not follow the correct format.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid SSN');
      }
      break;
    case 'AlphaOnly':
      var isValid = true;
      errorMsg += 'contain alpha characters only and cannot be null. The value in this message
"' + fieldValue + '" does not follow the correct format.';
      if (checkNull) {
        if ((fieldValue == null) || (fieldValue == '')) { isValid = false; }
      }
      if (fieldValue != null) {
        if (fieldValue.match(/^([a-zA-Z]+)$/)) == null) { isValid = false; }
      }

      if (!isValid) { fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, path
HL7, errorMsg, 'CONFORMANCE', 'Invalid format - alpha'); }
      break;
  }
}

```

## Javascript

```

case 'Numeric':
  var isValid = true;
  var regex = new RegExp("^((\\d){ " + fieldReqLength.toString() + "})$");

  if (fieldReqLength != '') {
    if ((fieldValue.match(regex) == null) || (fieldValue == invalidPattern)) {
      isValid = false;
      errorMsg += 'be a length of ' + fieldReqLength.toString() + ', must ';
    }
  }
  else {
    if (fieldValue.match(/\\d*/) == null) { isValid = false; }
  }
  errorMsg += 'be numeric with no special characters, cannot be all the same number and cannot be "' + invalidPattern + '". The value in this message "' + fieldValue + '" does not follow the correct format.';
  if (!isValid) {
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CONFORMANCE', 'Invalid format - numeric'); }
  break;
case 'Email':
  errorMsg += 'be a valid email format (localpart@domain). The value in this message "' + fieldValue + '" does not follow the correct format.';
  if (fieldValue.match(/^\\w+@[a-zA-Z_]+?\\. [a-zA-Z]{2,3}$/) == null) {
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CONFORMANCE', 'Invalid format - email');
  }
  break;
} // -----(END) switch(validationFormat) (END)
} // -----(END) if (fieldValue != null) (END)
}

//FUNCTION: fnOBX4Required
//Function to determine if OBX-4 is a required field.
function fnOBX4Required (pathHL7, fieldValue, fieldIndex) {
  var arrOBX3_1 = [];
  for(var k=0; k < ELRMessage.getRepeatCount(pathHL7 + '/OBSERVATION'); k++) {
    var OBX = pathHL7 + '/OBSERVATION[' + k + ']/OBX';
    if (k != fieldIndex) { arrOBX3_1.push(ELRMessage.getField(OBX + '/ObservationIdentifier/Identifier')); }
  }
  if (arrOBX3_1.indexOf(fieldValue) != -1) {
    return true;
  }
}

//FUNCTION: fnInfoNotTreatedAsRepeatable
//Function to create an info message if a field has repeating values and is not treated as repeatable.
function fnInfoNotTreatedAsRepeatable (fieldNotation, pathHL7) {
  if ((ELRMessage.getField(pathHL7) != null) && (ELRMessage.getRepeatCount(pathHL7) > 1)) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') has multiple values. NBS does not treat this field as repeatable. If more than one value is sent in the message, then only the first occurrence is stored.';
    fnCreateXMLERRecord ('NBS', 'INFO', fieldNotation, pathHL7, errorMsg, 'ODS PROCESSING COMMENT', 'Extra data');
  }
}

```

## Javascript

```

}
}

//FUNCTION: fnCreateErrorMsg
//Function to create an error message based on the definition error code.
function fnCreateErrorMsg (errorCode, errorInitial, errorPath, fieldNotation) {
  var returnMsg = '';
  switch(errorCode) {
    case '1001': //Required segment missing
      returnMsg = 'This message is missing ' + errorPath.substr((errorPath.lastIndexOf('/')+1), 3)
+ ' which is a required segment.';
      break;
    case '1006': //Required field missing
      returnMsg = 'This message is missing ' + fieldNotation + ' (' + errorPath.substr(errorPath
.lastIndexOf('/')+1) + ') which is a required field.';
      break;
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      if (ELRMessage.getField(errorPath) == null) { var fieldLength = 0; }
      else { var fieldLength = ELRMessage.getField(errorPath).length; }
      returnMsg = fieldNotation + ' (' + errorPath.substr(errorPath.lastIndexOf('/')+1) + ') has a
max limit of ' + errorInitial.substr(errorInitial.indexOf('>')+2) + ' characters. The value in
this message "'
+ ELRMessage.getField(errorPath) + '" is ' + fieldLength + ' characters.';
      break;
    case '1014': //Field value not found in validation table
      returnMsg = 'The code/value in field ' + fieldNotation + ' (' + errorPath.substr(errorPath
.lastIndexOf('/')+1)
+ ') was not found in the corresponding validation table. The code/value in this
message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1009': //Extra data found after field
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] |
[[/]] [A-Z] {2} \d [[]]);
      returnMsg = 'There is extra data in the ' + errorPath.substr(indexSegment+1, 3) + ' segment
found after the field "' + errorPath.substr(errorPath.lastIndexOf('/')+1) + '". The data in
this message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1010': //Extra data found after composite
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] |
[[/]] [A-Z] {2} \d [[]]);
      returnMsg = 'There is extra data in the ' + errorPath.substr(indexSegment+1, 3) + ' segment
for the field "' + errorPath.substr(errorPath.lastIndexOf('/')+1) + '". The data in this
message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1022': //Extra data found after segment
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] |
[[/]] [A-Z] {2} \d [[]]);
      returnMsg = 'There is extra data at the end of the ' + errorPath.substr(errorPath.last
IndexOf('/')+1, 3) + ' segment.';
      break;
    default:
      returnMsg = 'This message has a definition violation of ' + errorInitial + '.';
      break;
  }
  return returnMsg;
}

```

## Javascript

```

//FUNCTION: fnCreateErrorTypeDetail
//Function to create an error type detail based on the definition error code.
function fnCreateErrorTypeDetail (errorCode) {
  var returnDetail = '';
  switch(errorCode) {
    case '1001': //Required segment missing
    case '1006': //Required field missing
      returnDetail = 'Missing data';
      break;
    case '1007': //Field too short
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      returnDetail = 'Field length';
      break;
    case '1014': //Field value not found in validation table
      returnDetail = 'Invalid value';
      break;
    case '1009': //Extra data found after field
    case '1010': //Extra data found after composite
    case '1022': //Extra data found after segment
      returnDetail = 'Extra data';
      break;
    default:
      returnDetail = 'Other';
      break;
  }
  return returnDetail;
}

//FUNCTION: fnCreateErrorCategory
//Function to create an error category based on the definition error code.
function fnCreateErrorCategory (errorCode) {
  var returnCategory = '';
  switch(errorCode) {
    case '1001': //Required segment missing
      returnCategory = 'FATAL';
      break;
    case '1006': //Required field missing
    case '1007': //Field too short
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      returnCategory = 'WARNING';
      break;
    default:
      returnCategory = 'INFO';
      break;
  }
  return returnCategory;
}
//
=====
//_____ FUNCTIONS (end
)_____

```

## Javascript

```

//
=====

//
=====
//_____ NIST Definition Errors (start
)_____
//
=====
//Parse out all validation errors from the NIST Definition ACK generated and create an ELR
_ERRORS db record.

if (ackNIST.getField('ERR') != null) {
  for (var i=0; i < ackNIST.getRepeatCount('ERR'); i++) {
    var errFieldLocation = '';
    if (ackNIST.getField('ERR[' + i + ']/ErrorLocation') != null) {
      for (var j=0; j < ackNIST.getRepeatCount('ERR[' + i + ']/ErrorLocation'); j++) {
        errFieldLocation = ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SegmentID');
        if (ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/FieldPosition') != null)
        { errFieldLocation += '-' + ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/Field
Position'); }
        if (ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/ComponentNumber') != null)
        { errFieldLocation += '.' + ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/Com
ponentNumber'); }
        if (ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SubComponentNumber') != null)
        { errFieldLocation += '.' + ackNIST.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SubComp
onentNumber'); }
      }
    }
    var errPathHL7 = '';
    for (var j=0; j < ackNIST.getRepeatCount('ERR[' + i + ']/ApplicationErrorParameter'); j++) {
      if (errPathHL7 == '') { errPathHL7 = ackNIST.getField('ERR[' + i + ']/ApplicationErrorPara
meter[' + j + ']); }
      else { errPathHL7 += ', ' + ackNIST.getField('ERR[' + i + ']/ApplicationErrorParameter[' +
j + ']); }
    }
    var errorTypeDetail = fnCreateErrorTypeDetail(ackNIST.getField('ERR[' + i + ']/Application
ErrorCode/Identifier'));
    var errorMsg = fnCreateErrorMsg (ackNIST.getField('ERR[' + i + ']/ApplicationErrorCode/Ide
ntifier'),
      ackNIST.getField('ERR[' + i + ']/ApplicationErrorCode/Text'),
      errPathHL7, errFieldLocation);
    fnCreateXMLRecord ('NIST', 'INFO', errFieldLocation, errPathHL7, errorMsg, 'DEFINITION',
errorTypeDetail);
  }
}

//
=====
//_____ NIST Definition Errors (end
)_____
//
=====

```

## Javascript

```
//
=====
//_____ NIST Predicates and Conformance (start
)_____
//
=====
//Validate if all conditional fields have had conditions met
// and validate content of fields if a conformance rule exists.

//----- MSH segment -----

fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-1', 'MSH/FieldSeparator', '|', '', '');
fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-2', 'MSH/EncodingCharacters', '^~\\&#', '',
    '');
fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'MSH-3', 'MSH/SendingApplication');
fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'MSH-4', 'MSH/SendingFacility');
fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'MSH-5', 'MSH/ReceivingApplication');
fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'MSH-6', 'MSH/ReceivingFacility');
fnValidateDateTime ('TS_ELR_LD', 'NIST', 'INFO', 'MSH-7', 'MSH/DateTimeOfMessage');
fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-9.1', 'MSH/MessageType/MessageCode', 'ORU',
    '', '');
fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-9.2', 'MSH/MessageType/TriggerEvent', 'R01',
    '', '');
fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-9.3', 'MSH/MessageType/MessageStructure', 'ORU
R01', '', '');
fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-12.1', 'MSH/VersionID/VersionID', '2.5.1', '',
    '');
fnRequiredIfOtherField ('NIST', 'INFO', 'MSH-15', 'MSH-21', 'MSH/AcceptAcknowledgmentType',
    'MSH/MessageProfileIdentifier', 'PHLabReport-Ack', 'IsAnyValue');
fnRequiredIfOtherField ('NIST', 'INFO', 'MSH-16', 'MSH-21', 'MSH/ApplicationAcknowledgment
Type', 'MSH/MessageProfileIdentifier', 'PHLabReport-Ack', 'IsAnyValue');

//MSH-21 (MessageProfileIdentifier)
var hasValuePHLabReportAck = false;
var hasMSH_21_3_RequiredValue = false;
var hasMSH_21_1_RequiredValue = false;

for (var i=0; i < ELRMessage.getRepeatCount('MSH/MessageProfileIdentifier'); i++) {
    if (ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIdentifier') == 'PHLabR
eport-Ack') {
        hasValuePHLabReportAck = true;
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-15', 'MSH/AcceptAcknowledgmentType', 'AL',
            '', '');
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-16', 'MSH/ApplicationAcknowledgmentType',
            'SU', '', '');
    }

    //Conformance Rule ELR-22:MSH.21.3 (Universal ID) of an occurrence of MSH.21 (Message Profile
    Identifier) SHALL contain the value " 2.16.840.1.113883.9.11"
    if (ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/UniversalID') == '2.16.840.1
    .113883.9.11') { hasMSH_21_3_RequiredValue = true; }

    //Conformance Rule ELR-021:MSH.21.1 (Entity Identifier) of an occurrence of MSH.21 (Message
    Profile Identifier) SHALL be valued with 'PHLabReport-Ack' OR 'PHLabReport-NoAck' OR 'PHLabR
eport-Batch'
```

## Javascript

```

    if ((ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIdentifier') == 'PHLabReport-Ack') ||
        (ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIdentifier') == 'PHLabReport-NoAck') ||
        (ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIdentifier') == 'PHLabReport-Batch')) {
        hasMSH_21_1_RequiredValue = true;
    }

    fnValidateDataTypeRule ('EI_ELR', false, 'NIST', 'INFO', 'MSH-21', 'MSH/MessageProfileIdentifier[' + i + ']');
}

if ((!hasValuePHLabReportAck) && (ELRMessage.getField('MSH/MessageProfileIdentifier') != null))
{
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-15', 'MSH/AcceptAcknowledgmentType', 'NE', '', '');
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'MSH-16', 'MSH/ApplicationAcknowledgmentType', 'NE', '', '');
}

if (!hasMSH_21_3_RequiredValue) {
    var errorMsg = 'An occurrence of MSH-21.3 (MSH/MessageProfileIdentifier/UniversalID) must be the value "2.16.840.1.113883.9.11"';
    fnCreateXMLERRecord ('NIST', 'INFO', 'MSH-21.3', 'MSH/MessageProfileIdentifier[0]/UniversalID', errorMsg, 'CONFORMANCE', 'Field must equal value');
}

if (!hasMSH_21_1_RequiredValue) {
    var errorMsg = 'An occurrence of MSH-21.1 (MessageProfileIdentifier/EntityIdentifier) must be valued with "PHLabReport-Ack" OR "PHLabReport-NoAck" OR "PHLabReport-Batch"';
    fnCreateXMLERRecord ('NIST', 'INFO', 'MSH-21.1', 'MSH/MessageProfileIdentifier[0]/EntityIdentifier', errorMsg, 'CONFORMANCE', 'Field must equal value');
}

// (END) MSH-21 (MessageProfileIdentifier) (END)

//----- MSH segment (end) -----

//----- SFT segment -----

if (ELRMessage.getField('SFT') != null) {
    for (var i=0; i < ELRMessage.getRepeatCount('SFT'); i++) {
        fnValidateDataTypeRule ('XON_ELR', false, 'NIST', 'INFO', 'SFT-1', 'SFT[' + i + ']/SoftwareVendorOrganization');
        if (ELRMessage.getField('SFT[' + i + ']/SoftwareInstallDate') != null) { fnValidateDateTime ('TS_ELR_YO', 'NIST', 'INFO', 'SFT-6', 'SFT[' + i + ']/SoftwareInstallDate'); }
    }
}

//----- SFT segment (end) -----

//----- PATIENT_RESULT group -----

```

## Javascript

```

for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
  //----- PID segment -----

  var PID = 'PATIENT_RESULT[' + i + ']/PATIENT/PID';
  fnValidateFieldEqualsValue ('NIST', 'INFO', 'PID-1', PID + '/SetIDPID', '1', '', '');

  for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'PID-3.4', PID + '/PatientIdentifier
List[' + j + ']/AssigningAuthority');
    if (ELRMessage.getField(PID + '/PatientIdentifierList[' + j + ']/AssigningFacility') != null)
    { fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'PID-3.6', PID + '/PatientIden
tifierList[' + j + ']/AssigningFacility'); }
  }

  if (ELRMessage.getField(PID + '/MothersMaidenName/NameTypeCode') != null) {
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'PID-6.7', PID + '/MothersMaidenName/NameType
Code', 'M', '', '');
  }
  if (ELRMessage.getField(PID + '/DateTimeOfBirth') != null) { fnValidateDateTime ('TS_ELR
_YO', 'NIST', 'INFO', 'PID-7', PID + '/DateTimeOfBirth'); }

  if (ELRMessage.getField(PID + '/Race') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/Race'); j++) {
      fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'PID-10', PID + '/Race[' + j +
      ']);
    }
  }

  if (ELRMessage.getField(PID + '/PatientAddress/ZiporPostalCode') != null) { fnValidateFormat
('NIST', 'INFO', 'PID-11.5', PID + '/PatientAddress/ZiporPostalCode', '', '', '', 'ZipCode');
  }

  if (ELRMessage.getField(PID + '/PhoneNumberHome') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PhoneNumberHome'); j++) {
      fnValidateDataTypeRule ('XTN_ELR', false, 'NIST', 'INFO', 'PID-13', PID + '/PhoneNumber
Home[' + j + ']);
    }
  }

  if (ELRMessage.getField(PID + '/PhoneNumberBusiness') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PhoneNumberBusiness'); j++) {
      fnValidateDataTypeRule ('XTN_ELR', false, 'NIST', 'INFO', 'PID-14', PID + '/PhoneNumberBusi
ness[' + j + ']);
    }
  }

  if (ELRMessage.getField(PID + '/EthnicGroup') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/EthnicGroup'); j++) {
      fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'PID-22', PID + '/EthnicGroup[' +
      j + ']);
    }
  }
  if (ELRMessage.getField(PID + '/PatientDeathDateandTime') != null) { fnValidateDateTime ('TS
_ELR_YO', 'NIST', 'INFO', 'PID-29', PID + '/PatientDeathDateandTime'); }
  if (ELRMessage.getField(PID + '/LastUpdateDateTime') != null) { fnValidateDateTime ('TS_ELR
_YO', 'NIST', 'INFO', 'PID-33', PID + '/LastUpdateDateTime'); }

```

## Javascript

```
//Conditional Predicate - PID-34: Required if PATIENT_RESULT[' + i + ']/PATIENT/PID.33 (Last
Update Date/Time) is valued.
fnRequiredIfOtherField ('NIST', 'INFO', 'PID-34', 'PID-33', PID + '/LastUpdateFacility', PID +
'/LastUpdateDateTime', '', 'IsAnyValue');
if (ELRMessage.getField(PID + '/LastUpdateFacility') != null) { fnValidateDataTypeRule ('HD
_ELR', false, 'NIST', 'INFO', 'PID-34', PID + '/LastUpdateFacility'); }
if (ELRMessage.getField(PID + '/SpeciesCode') != null) { fnValidateDataTypeRule ('CWE_ELR',
false, 'NIST', 'INFO', 'PID-35', PID + '/SpeciesCode'); }

//----- PID segment (end) -----

//----- PID_NTE segment -----
if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NTE') != null) {
  for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NTE'); j++) {
    var PID_NTE = 'PATIENT_RESULT[' + i + ']/PATIENT/NTE[' + j + ']';
    var setIDPID_NTE = j + 1;
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'NTE-1', PID_NTE + '/SetIDNTE', setIDPID_NTE.to
String(), '', '');
    if (ELRMessage.getField(PID_NTE + '/CommentType') != null) { fnValidateDataTypeRule ('CWE
_ELR', false, 'NIST', 'INFO', 'NTE-4', PID_NTE + '/CommentType'); }
  }
}
//----- PID_NTE segment (end) -----

//----- NK1 segment -----
if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NK1') != null) {
  if (ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1') > 0) {
    for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1'); j++) {
      var NK1 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']';
      var setIDNK1 = j + 1;
      fnValidateFieldEqualsValue ('NIST', 'INFO', 'NK1-1', NK1 + '/SetIDNK1', setIDNK1.toString
(), '', '');
      fnRequiredIfOtherField ('NIST', 'INFO', 'NK1-2', 'NK1-13', NK1 + '/Name', NK1 + '/Organiz
ationNameNK1', '', 'IsNotValued');
      if (ELRMessage.getField(NK1 + '/Relationship') != null) { fnValidateDataTypeRule ('CWE
_ELR', false, 'NIST', 'INFO', 'NK1-3', NK1 + '/Relationship'); }
      if (ELRMessage.getField(NK1 + '/Address/ZiporPostalCode') != null) { fnValidateFormat
('NIST', 'INFO', 'NK1-4.5', NK1 + '/Address/ZiporPostalCode', '', '', '', 'ZipCode'); }

      if (ELRMessage.getField(NK1 + '/PhoneNumber') != null) {
        for(var k=0; k < ELRMessage.getRepeatCount(NK1 + '/PhoneNumber'); k++) {
          fnValidateDataTypeRule ('XTN_ELR', false, 'NIST', 'INFO', 'NK1-5', NK1 + '/PhoneNumber['
+ k + ']');
        }
      }

      fnRequiredIfOtherField ('NIST', 'INFO', 'NK1-13', 'NK1-2', NK1 + '/OrganizationNameNK1',
NK1 + '/Name', '', 'IsNotValued');
      if (ELRMessage.getField(NK1 + '/OrganizationNameNK1') != null) { fnValidateDataTypeRule
('XON_ELR', false, 'NIST', 'INFO', 'NK1-13', NK1 + '/OrganizationNameNK1'); }
      fnRequiredIfOtherField ('NIST', 'INFO', 'NK1-30', 'NK1-13', NK1 + '/ContactPersonsName',
NK1 + '/OrganizationNameNK1', '', 'IsAnyValue');

      if (ELRMessage.getField(NK1 + '/ContactPersonsTelephoneNumber') != null) {
        for(var k=0; k < ELRMessage.getRepeatCount(NK1 + '/ContactPersonsTelephoneNumber'); k++) {
```

## Javascript

```

        fnValidateDataTypeRule ('XTN_ELRL', false, 'NIST', 'INFO', 'NK1-31', NK1 + '/Contact
PersonsTelephoneNumber[' + k + ']');
    }
}
if (ELRMessage.getField(NK1 + '/ContactPersonsAddress/ZiporPostalCode') != null) {
    for(var k=0; k < ELRMessage.getRepeatCount(NK1 + '/ContactPersonsAddress'); k++) {
        fnValidateFormat ('NIST', 'INFO', 'NK1-32.5', NK1 + '/ContactPersonsAddress[' + k + ']/
ZiporPostalCode', '', '', '', 'ZipCode');
    }
}
}
}
}
//----- NK1 segment (end) -----

//----- PV1 segment -----
if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/VISIT/PV1/SetIDPV1') != null) {
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'PV1-1', 'PATIENT_RESULT[' + i + ']/PATIENT/VISIT
/PV1/SetIDPV1', '1', '', '');
    if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/VISIT/PV1/AdmitDateTime') != null)
    { fnValidateDateTime ('TS_ELRL_YO', 'NIST', 'INFO', 'PV1-44', 'PATIENT_RESULT[' + i + ']/PAT
IENT/VISIT/PV1/AdmitDateTime'); }
    if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/VISIT/PV1/DischargeDateTime') !=
null) { fnValidateDateTime ('TS_ELRL_YO', 'NIST', 'INFO', 'PV1-45', 'PATIENT_RESULT[' + i +
']/PATIENT/VISIT/PV1/DischargeDateTime'); }
}
//----- PV1 segment (end) -----

//---ORDER_OBSERVATION group---
var grpsPECIMENispresent = false;
var arrFillerOrderNumber = [];
for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION'); j+
+) {
    var ORC = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/ORC';
    var OBR = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBR';
    var SPM = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPM';
    //----- ORC segment -----
    if (ELRMessage.getField(ORC) != null) {
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'ORC-1', ORC + '/OrderControl', 'RE', '', '');
        fnRequiredIfOtherField ('NIST', 'INFO', 'ORC-2', 'OBR-2', ORC + '/PlacerOrderNumber', OBR +
'/PlacerOrderNumber', ELRMessage.getField(OBR + '/PlacerOrderNumber'), 'IsAnyValue');
        if (ELRMessage.getField(ORC + '/PlacerOrderNumber') != null) {
            fnValidateFieldEqualsValue ('NIST', 'INFO', 'ORC-2', ORC + '/PlacerOrderNumber', ELRMessage
.getField(OBR + '/PlacerOrderNumber'), 'OBR-2', OBR + '/PlacerOrderNumber');
        }
        if (ELRMessage.getField(ORC + '/PlacerOrderNumber') != null) { fnValidateDataTypeRule ('EI
_ELRL', false, 'NIST', 'INFO', 'ORC-2', ORC + '/PlacerOrderNumber'); }

        fnValidateFieldEqualsValue ('NIST', 'INFO', 'ORC-3', ORC + '/FillerOrderNumber', ELRMessage
.getField(OBR + '/FillerOrderNumber'), 'OBR-3', OBR + '/FillerOrderNumber');
        fnValidateDataTypeRule ('EI_ELRL', false, 'NIST', 'INFO', 'ORC-3', ORC + '/FillerOrderNumber
');
        if (ELRMessage.getField(ORC + '/PlacerGroupNumber') != null) { fnValidateDataTypeRule ('EI
_ELRL', false, 'NIST', 'INFO', 'ORC-4', ORC + '/PlacerGroupNumber'); }
    }
}

```

## Javascript

```

    fnRequiredIfOtherField ('NIST', 'INFO', 'ORC-12', 'OBR-16', ORC + '/OrderingProvider', OBR +
'/OrderingProvider', ELRMessage.getField(OBR + '/OrderingProvider'), 'IsAnyValue');
    if (ELRMessage.getField(ORC + '/OrderingProvider') != null) {
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'ORC-12', ORC + '/OrderingProvider', ELRMessage
.getField(OBR + '/OrderingProvider'), 'OBR-16', OBR + '/OrderingProvider');
        for(var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProvider'); k++) {
            fnValidateDataTypeRule ('XCN_ELR', false, 'NIST', 'INFO', 'ORC-12', ORC + '/OrderingProv
ider[' + k + ']');
        }
    }
    fnRequiredIfOtherField ('NIST', 'INFO', 'ORC-14', 'OBR-17', ORC + '/CallBackPhoneNumber',
OBR + '/OrderCallbackPhoneNumber', ELRMessage.getField(OBR + '/OrderCallbackPhoneNumber'), 'Is
AnyValue');

    if (ELRMessage.getField(ORC + '/CallBackPhoneNumber') != null) {
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'ORC-14', ORC + '/CallBackPhoneNumber', ELRM
essage.getField(OBR + '/OrderCallbackPhoneNumber'), 'OBR-17', OBR + '/OrderCallbackPhoneNumber
');
        for(var k=0; k < ELRMessage.getRepeatCount(ORC + '/CallBackPhoneNumber'); k++) {
            fnValidateDataTypeRule ('XTN_ELR', false, 'NIST', 'INFO', 'ORC-14', ORC + '/CallBackPhone
Number[' + k + ']');
        }
    }

    fnValidateDataTypeRule ('XON_ELR', false, 'NIST', 'INFO', 'ORC-21', ORC + '/OrderingFacility
Name');
    fnValidateFormat ('NIST', 'INFO', 'ORC-22.5', ORC + '/OrderingFacilityAddress/ZiporPostal
Code', '', '', '', 'ZipCode');

    for(var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingFacilityPhoneNumber'); k++) {
        fnValidateDataTypeRule ('XTN_ELR', false, 'NIST', 'INFO', 'ORC-23', ORC + '/OrderingFaci
lityPhoneNumber[' + k + ']');
    }

    for(var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProviderAddress'); k++) {
        fnValidateFormat ('NIST', 'INFO', 'ORC-24.5', ORC + '/OrderingProviderAddress[' + k + ']/
ZiporPostalCode', '', '', '', 'ZipCode');
    }

} // -----(END) if (ELRMessage.getField(ORC) != null) (END)
//----- ORC segment (end) -----

//----- OBR segment -----
var setIDOBR = j + 1;
fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBR-1', OBR + '/SetIDOBR', setIDOBR.toString(),
'', '');
if (ELRMessage.getField(OBR + '/PlacerOrderNumber') != null) { fnValidateDataTypeRule ('EI
_ELR', false, 'NIST', 'INFO', 'OBR-2', OBR + '/PlacerOrderNumber'); }

fnValidateDataTypeRule ('EI_ELR', false, 'NIST', 'INFO', 'OBR-3', OBR + '/FillerOrderNumber
');
arrFillerOrderNumber[j] = ELRMessage.getField(OBR + '/FillerOrderNumber');

fnValidateDataTypeRule ('CWE_ELR', true, 'NIST', 'INFO', 'OBR-4', OBR + '/UniversalService
Identifier');
fnValidateDateTime ('TS_ELR_0000', 'NIST', 'INFO', 'OBR-7', OBR + '/ObservationDateTime');

```

## Javascript

```

fnRequiredIfOtherField ('NIST', 'INFO', 'OBR-8', 'SPM-17.2', OBR + '/ObservationEndDateTime',
SPM + '/SpecimenCollectionDateTime/RangeEndDateTime', '', 'IsAnyValue');
if (ELRMessage.getField(OBR + '/ObservationEndDateTime') != null) { fnValidateDateTime ('TS
ELR_0000', 'NIST', 'INFO', 'OBR-8', OBR + '/ObservationEndDateTime'); }
if (ELRMessage.getField(OBR + '/OrderingProvider') != null) {
for(var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderingProvider'); k++) {
fnValidateDataTypeRule ('XCN_ELRL', false, 'NIST', 'INFO', 'OBR-16', OBR + '/OrderingProv
ider[' + k + ']);
}
}
if (ELRMessage.getField(OBR + '/OrderCallbackPhoneNumber') != null) {
for(var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderCallbackPhoneNumber'); k++) {
fnValidateDataTypeRule ('XTN_ELRL', false, 'NIST', 'INFO', 'OBR-17', OBR + '/OrderCallback
PhoneNumber[' + k + ']);
}
}

fnValidateDateTime ('TS_ELRL_SecOpt', 'NIST', 'INFO', 'OBR-22', OBR + '/ResultsRptStatusChng
DateTime');

if (ELRMessage.getField(OBR + '/ParentResult') != null) { fnValidateDataTypeRule ('CWE
ELRL', true, 'NIST', 'INFO', 'OBR-26.1', OBR + '/ParentResult/ParentObservationIdentifier'); }
if (ELRMessage.getField(OBR + '/Parent') != null) { fnValidateDataTypeRule ('EI_ELRL',
false, 'NIST', 'INFO', 'OBR-29.2', OBR + '/Parent/FillerAssignedIdentifier'); }

if (ELRMessage.getField(OBR + '/ReasonforStudy') != null) {
for(var k=0; k < ELRMessage.getRepeatCount(OBR + '/ReasonforStudy'); k++) {
fnValidateDataTypeRule ('CWE_ELRL', true, 'NIST', 'INFO', 'OBR-31', OBR + '/Reasonfor
Study[' + k + ']);
}
}
//----- OBR segment (end) -----

//----- OBR_NTE segment -----
if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/NTE') !=
null) {
for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' +
j + ']/NTE'); k++) {
var OBR_NTE = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/NTE[' + k + '];
var setIDOBR_NTE = k + 1;
fnValidateFieldEqualsValue ('NIST', 'INFO', 'NTE-1', OBR_NTE + '/SetIDNTE', setIDOBR_NTE.to
String(), '', '');
if (ELRMessage.getField(OBR_NTE + '/CommentType') != null) { fnValidateDataTypeRule ('CWE
ELRL', false, 'NIST', 'INFO', 'NTE-4', OBR_NTE + '/CommentType'); }
}
}
//----- OBR_NTE segment (end) -----

//----- OBSERVATION group -----
//Conditional Predicate - OBSERVATION: Required if OBR.25 (Result Status) is valued "A",
"C", "F", "P", or "R".
var rsltStatus = ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/
OBR/ResultStatus');

```

## Javascript

```

    if ((rsltStatus = 'A') || (rsltStatus = 'C') || (rsltStatus = 'F') || (rsltStatus = 'P')
    || (rsltStatus = 'R')) && ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVA
TION[' + j + ']/OBSERVATION') == 0) {
        var errorMsg = 'This message is missing an OBSERVATION/OBX segment which is required if
OBR-25 (Result Status) is valued "A", "C", "F", "P", or "R".';
        fnCreateXMLErrorRecord ('NIST', 'INFO', 'OBX', 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVA
TION[' + j + ']/OBSERVATION', errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
    }

    for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j
+ ']/OBSERVATION'); k++) {
        var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/
OBX';

        //----- OBX segment -----
        var setIDOBX = k + 1;
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBX-1', OBX + '/SetIDOBX', setIDOBX.toString(),
'', '');

        fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-2', 'OBX-5', OBX + '/ValueType', OBX + '/Obs
ervationValue', '', 'IsAnyValue');
        fnValidateDataTypeRule ('CWE_ELR', true, 'NIST', 'INFO', 'OBX-3', OBX + '/ObservationIden
tifier');

        if ((ELRMessage.getField(OBX + '/ObservationSubID') == null) && (fnOBX4Required('PATIENT
_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']', ELRMessage.getField(OBX + '/ObservationIden
tifier/Identifier'), k))) {
            var errorMsg = 'This message is missing OBX-4 (ObservationSubID) which is required if there
are multiple OBX segments associated with the same OBR segment that have the same OBX-3 values
for (OBX-3.1 and OBX-3.3) or (OBX-3.4 and OBX-3.6).';
            fnCreateXMLErrorRecord ('NIST', 'INFO', 'OBX-4', OBX + '/ObservationSubID', errorMsg, 'CON
DITIONAL PREDICATE', 'Missing data');
        }

        //OBX-5 (ObservationValue)
        if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {
            fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-5', 'OBX-8', OBX + '/ObservationValue', OBX +
'/AbnormalFlags', '', 'IsNotValued');
        }

        // Validation based on OBX-2 (ValueType)
        var valueType = ELRMessage.getField(OBX + '/ValueType');
        switch(valueType) {
            case 'CE':
            case 'CWE':
            case 'CWE_ELR':
                fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'OBX-5', OBX + '/Observation
Value');
                break;
            case 'ED':
                fnValidateDataTypeRule ('HD_ELR', false, 'NIST', 'INFO', 'OBX-5.1', OBX + '/Observation
Value/SourceApplication');
                break;
            case 'SN':
                if (ELRMessage.getField(OBX + '/ObservationValue/Comparator') != null) {
                    fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBX-5.1', OBX + '/ObservationValue/Com
parator', '>_||<_||>=_||<=_||<>', '', '');
                }
        }
    }

```

## Javascript

```

    if (ELRMessage.getField(OBX + '/ObservationValue/SeparatorSuffix') != null) {
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBX-5.3', OBX + '/ObservationValue/Separator
Suffix', '-_||+_||/_||_._||_: ', '', '');
    }
    break;
}

//Conditional Predicate - OBX-6: Required if OBX.2 (Value Type) is valued 'NM' , 'SN' AND
OBX.11 is not valued 'X'.
if (((ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') && ((ELRMessage.getField
(OBX + '/ValueType') == 'NM') || (ELRMessage.getField(OBX + '/ValueType') == 'SN')))) && (ELRM
essage.getField(OBX + '/Units') == null)) {
    var errorMsg = 'This message is missing OBX-6 (Units) which is a required field if OBX-2 is
valued "NM" or "SN" AND OBX-11 is not valued "X".';
    fnCreateXMLErrorRecord ('NIST', 'INFO', 'OBX-6', OBX + '/Units', errorMsg, 'CONDITIONAL PRE
DICATE', 'Missing data');
}

if (ELRMessage.getField(OBX + '/Units') != null) { fnValidateDataTypeRule ('CWE_ELR',
false, 'NIST', 'INFO', 'OBX-6', OBX + '/Units'); }

if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {
    fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-8', 'OBX-5', OBX + '/AbnormalFlags', OBX + '/'
ObservationValue', '', 'IsNotValued');
}
if (ELRMessage.getField(OBX + '/AbnormalFlags') != null) {
    for (var l=0; l < ELRMessage.getRepeatCount(OBX + '/AbnormalFlags'); l++) {
        fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'OBX-8', OBX + '/Abnormal
Flags[' + l + ']');
    }
}

if (ELRMessage.getField(OBX + '/DateTimeoftheObservation') != null) { fnValidateDateTime
('TS_ELR_0000', 'NIST', 'INFO', 'OBX-14', OBX + '/DateTimeoftheObservation'); }

if (ELRMessage.getField(OBX + '/ObservationMethod') != null) {
    for (var l=0; l < ELRMessage.getRepeatCount(OBX + '/ObservationMethod'); l++) {
        fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'OBX-17', OBX + '/Observation
Method[' + l + ']');
    }
}

if (ELRMessage.getField(OBX + '/DateTimeoftheAnalysis') != null) { fnValidateDateTime ('TS
-ELR_YO', 'NIST', 'INFO', 'OBX-19', OBX + '/DateTimeoftheAnalysis'); }
fnValidateDataTypeRule ('XON_ELR', false, 'NIST', 'INFO', 'OBX-23', OBX + '/PerformingOrga
nizationName');
fnValidateFormat ('NIST', 'INFO', 'OBX-24.5', OBX + '/PerformingOrganizationAddress/Zipor
PostalCode', '', '', '', 'ZipCode');
fnValidateDataTypeRule ('XCN_ELR', false, 'NIST', 'INFO', 'OBX-25', OBX + '/PerformingOrga
nizationMedicalDirector');
//----- OBX segment (end) -----

//----- OBX_NTE segment -----
if (ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBS
ERVATION[' + k + ']/NTE') > 0) {

```

## Javascript

```

    for (var l=0; l < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/NTE'); l++) {
        var OBX_NTE = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/NTE[' + l + ']';
        var setIDOBX_NTE = l + 1;
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'NTE-1', OBX_NTE + '/SetIDNTE', setIDOBX_NTE.toString(), '', '');

        if (ELRMessage.getField(OBX_NTE + '/CommentType') != null) { fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'NTE-4', OBX_NTE + '/CommentType'); }
    }
    //----- OBX_NTE segment (end) -----

} // -----(END) for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) (END)
//----- OBSERVATION group (end) -----

if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[0]') != null) {
    grpSPECIMENispresent = true;
    //----- SPECIMEN group ----- (NIST cardinality is only [0,1] even though HL7 Standards cardinality is [0,*].
    //----- SPM segment -----
    var SPM = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[0]/SPM';
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'SPM-1', SPM + '/SetIDSPM', '1', '', '');

    if (ELRMessage.getField(SPM + '/FillerAssignedIdentifier') != null) { fnValidateDataTypeRule ('EI_ELR', false, 'NIST', 'INFO', 'SPM-2.2', SPM + '/SpecimenID/FillerAssignedIdentifier'); }

    fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-4', SPM + '/SpecimenType');

    if (ELRMessage.getField(SPM + '/SpecimenTypeModifier') != null) {
        for (var k=0; k < ELRMessage.getRepeatCount(SPM + '/SpecimenTypeModifier'); k++) {
            fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-5', SPM + '/SpecimenTypeModifier[' + k + ']);
        }

        if (ELRMessage.getField(SPM + '/SpecimenAdditives') != null) {
            for (var k=0; k < ELRMessage.getRepeatCount(SPM + '/SpecimenAdditives'); k++) {
                fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-6', SPM + '/SpecimenAdditives[' + k + ']);
            }

            if (ELRMessage.getField(SPM + '/SpecimenCollectionMethod') != null) { fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-7', SPM + '/SpecimenCollectionMethod'); }
            if (ELRMessage.getField(SPM + '/SpecimenSourceSite') != null) { fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-8', SPM + '/SpecimenSourceSite'); }

            if (ELRMessage.getField(SPM + '/SpecimenSourceSiteModifier') != null) {
                for (var k=0; k < ELRMessage.getRepeatCount(SPM + '/SpecimenSourceSiteModifier'); k++) {

```

## Javascript

```

    fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-9', SPM + '/SpecimenSource
SiteModifier[' + k + ']);
    }
}

if (ELRMessage.getField(SPM + '/SpecimenRole') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(SPM + '/SpecimenRole'); k++) {
        fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-11', SPM + '/Specimen
Role[' + k + ']);
    }
}

if (ELRMessage.getField(SPM + '/SpecimenCollectionAmount/Units') != null) { fnValidateData
TypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-12.2', SPM + '/SpecimenCollectionAmount/Units
'); }

fnValidateFieldEqualsValue ('NIST', 'INFO', 'SPM-17.1', SPM + '/SpecimenCollectionDateTime
/RangeStartDateTime', ELRMessage.getField(OBR + '/ObservationDateTime'), 'OBR-7', OBR + '/Obs
ervationDateTime');
    if (ELRMessage.getField(SPM + '/SpecimenCollectionDateTime/RangeStartDateTime') != null) {
        fnValidateDateTime ('TS_ELR_0000', 'NIST', 'INFO', 'SPM-17.1', SPM + '/SpecimenCollectionDate
Time/RangeStartDateTime'); }
    fnValidateFieldEqualsValue ('NIST', 'INFO', 'SPM-17.2', SPM + '/SpecimenCollectionDateTime
/RangeEndDateTime', ELRMessage.getField(OBR + '/ObservationEndDateTime'), 'OBR.8', OBR + '/Obs
ervationEndDateTime');
    if (ELRMessage.getField(SPM + '/SpecimenCollectionDateTime/RangeEndDateTime') != null) {
        fnValidateDateTime ('TS_ELR_0000', 'NIST', 'INFO', 'SPM-17.2', SPM + '/SpecimenCollectionDate
Time/RangeEndDateTime'); }
    fnValidateDateTime ('TS_ELR_TimeOpt', 'NIST', 'INFO', 'SPM-18', SPM + '/SpecimenReceivedDate
Time');

    if (ELRMessage.getField(SPM + '/SpecimenRejectReason') != null) {
        for (var k=0; k < ELRMessage.getRepeatCount(SPM + '/SpecimenRejectReason'); k++) {
            fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'SPM-21', SPM + '/Specimen
RejectReason[' + k + ']);
        }
    }
}
//----- SPM segment (end) -----

if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[0]/
OBX') != null) {
    //----- OBX segments -----
    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION['
+ j + ']/SPECIMEN[0]/OBX'); k++) {
        var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[0]/OBX[' + k +
']';

        var setIDOBX = k + 1;
        fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBX-1', OBX + '/SetIDOBX', setIDOBX.toString
(), '', '');

        fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-2', 'OBX-5', OBX + '/ValueType', OBX + '/Obs
ervationValue', '', 'IsAnyValue');
        fnValidateDataTypeRule ('CWE_ELR', true, 'NIST', 'INFO', 'OBX-3', OBX + '/ObservationIden
tifier');

        if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {

```

## Javascript

```

    fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-5', 'OBX-8', OBX + '/ObservationValue', OBX
+ '/AbnormalFlags', '', 'IsNotValued');
}

//Conditional Predicate - OBX-6: Required if OBX.2 (Value Type) is valued 'NM' , 'SN' AND
OBX.11 is not valued 'X'.
if (((ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') && (ELRMessage.get
Field(OBX + '/ValueType') == 'NM') || (ELRMessage.getField(OBX + '/ValueType') == 'SN')) &&
(ELRMessage.getField(OBX + '/Units') == null)) {
    var errorMsg = 'This message is missing OBX-6 (Units) which is a required field if OBX-2
is valued "NM" or "SN" AND OBX-11 is not valued "X".';
    fnCreateXMLRecord ('NIST', 'INFO', 'OBX-6', OBX + '/Units', errorMsg, 'CONDITIONAL
PREDICATE', 'Missing data');
}

if (ELRMessage.getField(OBX + '/Units') != null) { fnValidateDataTypeRule ('CWE_ELR',
false, 'NIST', 'INFO', 'OBX-6', OBX + '/Units'); }

if (ELRMessage.getField(OBX + '/ObservationResultStatus') != 'X') {
    fnRequiredIfOtherField ('NIST', 'INFO', 'OBX-8', 'OBX-5', OBX + '/AbnormalFlags', OBX +
'/ObservationValue', '', 'IsNotValued');
}
if (ELRMessage.getField(OBX + '/AbnormalFlags') != null) {
    for (var l=0; l < ELRMessage.getRepeatCount(OBX + '/AbnormalFlags'); l++) {
        fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'OBX-8', OBX + '/Abnormal
Flags');
    }
}

fnValidateFieldEqualsValue ('NIST', 'INFO', 'OBX-14', OBX + '/DateTimeoftheObservation',
ELRMessage.getField(OBR + '/ObservationDateTime', 'OBR-7', OBR + '/ObservationDateTime');

if (ELRMessage.getField(OBX + '/DateTimeoftheObservation') != null) { fnValidateDateTime
('TS_ELR_0000', 'NIST', 'INFO', 'OBX-14', OBX + '/DateTimeoftheObservation'); }

if (ELRMessage.getField(OBX + '/ObservationMethod') != null) {
    for (var l=0; l < ELRMessage.getRepeatCount(OBX + '/ObservationMethod'); l++) {
        fnValidateDataTypeRule ('CWE_ELR', false, 'NIST', 'INFO', 'OBX-17', OBX + '/Observation
Method[' + l + ']');
    }
}

if (ELRMessage.getField(OBX + '/DateTimeoftheAnalysis') != null) { fnValidateDateTime
('TS-ELR_YO', 'NIST', 'INFO', 'OBX-19', OBX + '/DateTimeoftheAnalysis'); }
fnValidateDataTypeRule ('XON_ELR', false, 'NIST', 'INFO', 'OBX-23', OBX + '/Performing
OrganizationName');
fnValidateFormat ('NIST', 'INFO', 'OBX-24.5', OBX + '/PerformingOrganizationAddress/Zipor
PostalCode', '', '', '', 'ZipCode');
fnValidateDataTypeRule ('XCN_ELR', false, 'NIST', 'INFO', 'OBX-25', OBX + '/Performing
OrganizationMedicalDirector');
}
//----- OBX segments (end) -----
}

//----- SPECIMEN group (end) -----

} // -----(END) if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j
+ ']/SPECIMEN[0]') != null) (END)

```

## Javascript

```

} // -----(END) for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION'); j++) (END)
//--- ORDER_OBSERVATION group (end) ---

//ELR-040:OBR.3 (Filler Order Number) SHALL NOT contain the same value as another occurrence
of OBR.3 (Filler Order Number) in the message.
if (arrFillerOrderNumber.length >=2) {
  for (var j=0; j < arrFillerOrderNumber.length; j++) {
    for (var k=j+1; k < arrFillerOrderNumber.length; k++) {
      if (arrFillerOrderNumber[j] == arrFillerOrderNumber[k]) {
        var errorMsg = 'This field contains the same value as another occurrence of OBR-3 (Filler
OrderNumber) in the message.';
        fnCreateXMLErrorRecord ('NIST', 'INFO', 'OBR-3', 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVA
TION[' + j + ']/OBR/FillerOrderNumber', errorMsg, 'CONFORMANCE', 'Not unique');
      }
    }
  }
}

//ELR-064:Specimen (Specimen Group) SHALL be present in at least one occurrence of one Order
Observation Group.
if (!grpSPECIMENispresent) {
  var errorMsg = 'This message is missing the SPECIMEN GROUP which is required in at least one
occurrence of one Order_Observation Group.';
  fnCreateXMLErrorRecord ('NIST', 'INFO', 'SPM', 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[0
]/SPECIMEN[0]', errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
}

} // -----(END) for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
//----- PATIENT_RESULT group (end) -----

//
=====
//_____ NIST Predicates and Conformance (end
)_____
//
=====

//
=====
//_____ NBS Definition Errors (start
)_____
//
=====
//Parse out all validation errors from the NBS Definition ACK generated and create an ELR
_ERRORS db record.
next.text = dbInsertsXML;

//NHBS messages are not sent to NBS.
if (next.getProperty('nhbs_SendingFac') != null) { var nhbs_SendingFac = next.getProperty
('nhbs_SendingFac').toLowerCase(); }

```

## Javascript

```

if (next.getProperty('nhbs_PatientLName') != null) { var nhbs_PatientLName = next.getProperty('nhbs_PatientLName').toLowerCase(); }

if (!(nhbs_SendingFac == '██████████') && (nhbs_PatientLName == 'nhbs')) {

  if (ackNBS.getField('ERR') != null) {
    for (var i=0; i < ackNBS.getRepeatCount('ERR'); i++) {
      var errFieldLocation = '';
      if (ackNBS.getField('ERR[' + i + ']/ErrorLocation') != null) {
        for (var j=0; j < ackNBS.getRepeatCount('ERR[' + i + ']/ErrorLocation'); j++) {
          errFieldLocation = ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SegmentID');
          if (ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/FieldPosition') != null)
            { errFieldLocation += '-' + ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/FieldPosition'); }
          if (ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/ComponentNumber') != null)
            { errFieldLocation += '.' + ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/ComponentNumber'); }
          if (ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SubComponentNumber') != null)
            { errFieldLocation += '.' + ackNBS.getField('ERR[' + i + ']/ErrorLocation[' + j + ']/SubComponentNumber'); }
        }
      }
      var errPathHL7 = '';
      for (var j=0; j < ackNBS.getRepeatCount('ERR[' + i + ']/ApplicationErrorParameter'); j++) {
        if (errPathHL7 == '') { errPathHL7 = ackNBS.getField('ERR[' + i + ']/ApplicationErrorParameter[' + j + ']); }
        else { errPathHL7 += ', ' + ackNBS.getField('ERR[' + i + ']/ApplicationErrorParameter[' + j + ']); }
      }
      var errorCategory = fnCreateErrorCategory(ackNBS.getField('ERR[' + i + ']/ApplicationErrorCode/Identifier'));
      //Some specific fields are FATAL errors instead of being based on the error code.
      switch(errFieldLocation) {
        case 'MSH-1':
        case 'MSH-2':
        case 'MSH-4':
        case 'MSH-10':
        case 'OBR-3':
          errorCategory = 'FATAL';
          break;
      }
      var errorTypeDetail = fnCreateErrorTypeDetail(ackNBS.getField('ERR[' + i + ']/ApplicationErrorCode/Identifier'));
      var errorMsg = fnCreateErrorMsg(ackNBS.getField('ERR[' + i + ']/ApplicationErrorCode/Identifier'),
        ackNBS.getField('ERR[' + i + ']/ApplicationErrorCode/Text'),
        errPathHL7, errFieldLocation);

      fnCheckForSimilarErrors ('NBS', errorCategory, errFieldLocation, errPathHL7, errorMsg, 'DEFINITION', errorTypeDetail);
    }
  }
}

//
=====
//_____ NBS Definition Errors (end
)_____

```

## Javascript

```
//
=====

//
=====
// _____ NBS Predicates and Conformance (start
)
//
=====

//Validate if all conditional fields have had conditions met
// and validate content of fields if a conformance rule exists.

//NHBS messages are not sent to NBS.
if (next.getProperty('nhbs_SendingFac') != null) { var nhbs_SendingFac = next.getProperty
('nhbs_SendingFac').toLowerCase(); }
if (next.getProperty('nhbs_PatientLName') != null) { var nhbs_PatientLName = next.getProperty
('nhbs_PatientLName').toLowerCase(); }

if (!(nhbs_SendingFac == 'cphl' && (nhbs_PatientLName == 'nhbs'))) {

//----- PATIENT_RESULT group -----
for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
var PID = 'PATIENT_RESULT[' + i + ']/PATIENT/PID';

//----- PID segment -----
fnInfoNotTreatedAsRepeatable ('PID-13', PID + '/PhoneNumberHome');
fnInfoNotTreatedAsRepeatable ('PID-14', PID + '/PhoneNumberBusiness');
fnInfoNotTreatedAsRepeatable ('PID-15', PID + '/PrimaryLanguage');
fnInfoNotTreatedAsRepeatable ('PID-21', PID + '/MothersIdentifier');
//----- PID segment (end) -----

if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NK1') != null) {
for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1'); j++) {
var NK1 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + '];

//----- NK1 segment -----
fnInfoNotTreatedAsRepeatable ('NK1-2', NK1 + '/Name');
fnInfoNotTreatedAsRepeatable ('NK1-3.1', NK1 + '/Relationship/Identifier');
fnInfoNotTreatedAsRepeatable ('NK1-4', NK1 + '/Address');
fnInfoNotTreatedAsRepeatable ('NK1-5', NK1 + '/PhoneNumber');
//----- NK1 segment (end) -----
}
}

//----- ORDER_OBSERVATION group -----
for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION'); j
++) {
var ORC = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/ORC';
var OBR = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBR';
var SPM = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPM';

//----- ORC segment -----
```

## Javascript

```

    if ((ELRMessage.getField(ORC + '/OrderingProvider') == null) && (ELRMessage.getField(ORC +
'/OrderingFacilityName') == null)) {
        var errorMsg = 'Either Ordering Facility (ORC-21) or Ordering Provider (ORC-12) must be
present in the message. Both fields in the message are empty.';
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-12', ORC + '/OrderingProvider', errorMsg,
'CONDITIONAL PREDICATE', 'Missing data');
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-21', ORC + '/OrderingFacilityName', error
Msg, 'CONDITIONAL PREDICATE', 'Missing data');
    }
    fnInfoNotTreatedAsRepeatable ('ORC-21', ORC + '/OrderingFacilityName');
    fnInfoNotTreatedAsRepeatable ('ORC-22', ORC + '/OrderingFacilityAddress');
    fnInfoNotTreatedAsRepeatable ('ORC-23', ORC + '/OrderingFacilityPhoneNumber');
    fnInfoNotTreatedAsRepeatable ('ORC-24', ORC + '/OrderingProviderAddress');
    //----- ORC segment (end) -----

    //----- OBR segment -----
    fnInfoNotTreatedAsRepeatable ('OBR-10', OBR + '/CollectorIdentifier');
    if ((ELRMessage.getField(OBR + '/OrderingProvider') == null) && (ELRMessage.getField(ORC +
'/OrderingFacilityName') == null)) {
        var errorMsg = 'Either Ordering Facility (ORC-21) or Ordering Provider (OBR-16) must be
present in the message. Both fields in the message are empty.';
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'OBR-16', OBR + '/OrderingProvider', errorMsg,
'CONFORMANCE', 'Missing data');

        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-21', ORC + '/OrderingFacilityName', error
Msg, 'CONFORMANCE', 'Missing data');
    }
    fnInfoNotTreatedAsRepeatable ('OBR-16', OBR + '/OrderingProvider');
    fnInfoNotTreatedAsRepeatable ('OBR-17', OBR + '/OrderCallbackPhoneNumber');
    //----- OBR segment (end) -----

    //----- OBSERVATION group -----
    for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' +
j + ']/OBSERVATION'); k++) {
        var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/
OBX';

        //----- OBX segment -----
        if ((ELRMessage.getField(OBX + '/ObservationSubID') == null) && (fnOBX4Required('PATIENT
_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']', ELRMessage.getField(OBX + '/ObservationIden
tifier/Identifier'), k))) {
            var errorMsg = 'This message is missing OBX-4 (ObservationSubID) which is required if
there are multiple OBX segments associated with the same OBR segment that have the same OBX-3
values for (OBX-3.1 and OBX-3.3) or (OBX-3.4 and OBX-3.6).';
            fnCheckForSimilarErrors ('NBS', 'WARNING', 'OBX-4', OBX + '/ObservationSubID', errorMsg,
'CONDITIONAL PREDICATE', 'Missing data');
        }
        if ((ELRMessage.getField(OBX + '/Units') == null) && (ELRMessage.getField(OBX + '/Value
Type') == 'SN' || ELRMessage.getField(OBX + '/ValueType') == 'NM')) {
            var errorMsg = 'This message is missing OBX-6 (Units) which is a required field if OBX-2
is valued "NM" or "SN" AND OBX-11 is not valued "X".';
            fnCheckForSimilarErrors ('NBS', 'WARNING', 'OBX-6', OBX + '/Units', errorMsg, 'CON
DITIONAL PREDICATE', 'Missing data');
        }
        fnInfoNotTreatedAsRepeatable ('OBX-8', OBX + '/AbnormalFlags');
    }
    //----- OBX segment (end) -----

```

## Javascript

```

    } // ----- (END) for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + '']/
ORDER_OBSERVATION[' + j + '']/OBSERVATION'); k++) (END)

    //----- OBSERVATION group (end) -----

    //----- SPECIMEN group -----
    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + '']/ORDER_OBSERVATION[' +
j + '']/SPECIMEN'); k++) {
        for (var l=0; l < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + '']/ORDER_OBSERVATION[' +
j + '']/SPECIMEN[' + k + '']/OBSERVATION'); l++) {
            var OBX = 'PATIENT_RESULT[' + i + '']/ORDER_OBSERVATION[' + j + '']/SPECIMEN[' + k + '']/
OBX[' + l + ''];

            //----- OBX segment -----
            if ((ELRMessage.getField(OBX + '/Units') == null) && (ELRMessage.getField(OBX + '/Value
Type') == 'SN' || ELRMessage.getField(OBX + '/ValueType') == 'NM')) {
                var errorMsg = 'This message is missing OBX-6 (Units) which is a required field if OBX-2
is valued "NM" or "SN" AND OBX-11 is not valued "X".';
                fnCheckForSimilarErrors ('NBS', 'WARNING', 'OBX-6', OBX + '/Units', errorMsg, 'CON
DITIONAL PREDICATE', 'Missing data');
            }
            fnInfoNotTreatedAsRepeatable ('OBX-8', OBX + '/AbnormalFlags');
            //----- OBX segment (end) -----
        } // ----- (END)

    } // ----- (END) for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + '']/
ORDER_OBSERVATION[' + j + '']/SPECIMEN'); k++) (END)

    //----- SPECIMEN group (end) -----

    } // ----- (END) for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + '']/
ORDER_OBSERVATION'); j++) (END)
    //--- ORDER_OBSERVATION group (end) ---

    } // ----- (END) for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
    //--- PATIENT_RESULT group (end) ---
} // ----- (END) if (!((next.getProperty('nhbs_SendingFac').toLowerCase() == '
.cphl') && (next.getProperty('nhbs_PatientLName').toLowerCase() == 'nhbs')) (END)

//
=====
// _____ NBS Predicates and Conformance (end
)
//
=====

//
=====
// _____ Conformance Rules (start
)
//
=====
//Validate against all _____ conformance rules.

```

## Javascript

```
//----- MSH segment -----
fnValidateFieldEqualsValue(█, 'INFO', 'MSH-1', 'MSH/FieldSeparator', '|', '', '');
if (ELRMessage.getField('MSH/SendingFacility/UniversalIDType') == 'CLIA') { fnValidateFormat
('█' 'INFO', 'MSH-4.2', 'MSH/SendingFacility/UniversalID', '', '', '', 'CLIA'); }
if (ELRMessage.getField('MSH/SendingFacility/UniversalIDType') == 'ISO') { fnValidateFormat
('█' 'INFO', 'MSH-4.2', 'MSH/SendingFacility/UniversalID', '', '', '', 'ISO'); }
fnIsValidUniversalIDType(█, 'INFO', 'MSH-4.3', 'MSH/SendingFacility/UniversalIDType', 'ISO
CLIA');
fnValidateFieldEqualsValue(█, 'INFO', 'MSH-5', 'MSH/ReceivingApplication', █ELR^2.16
.840.1.113883.3.773.1.1.3^ISO', '', '');
fnValidateFieldEqualsValue(█, 'INFO', 'MSH-6', 'MSH/ReceivingFacility', █^2.16.840.1
.113883.3.773^ISO', '', '');
fnValidateWithinTimeFrame(█, 'INFO', 'MSH-7', 'MSH/DateTimeOfMessage', 30);

if (ELRMessage.getField('MSH/VersionID') == '2.5.1') {
  fnValidateFieldEqualsValue (█ 'INFO', 'MSH-9.1', 'MSH/MessageType/MessageCode', 'ORU',
  '', '');
  fnValidateFieldEqualsValue (█, 'INFO', 'MSH-9.2', 'MSH/MessageType/TriggerEvent', 'R01',
  '', '');
  fnValidateFieldEqualsValue (█, 'INFO', 'MSH-9.3', 'MSH/MessageType/MessageStructure', 'ORU
  R01', '', '');
}

if ((ELRMessage.getField('MSH/VersionID').indexOf('2.3.1') == -1) && (ELRMessage.getField('MSH/
VersionID').indexOf('2.5.1') == -1)) {
  var errorMsg = 'For MSH-12 (VersionID), "2.3.1" or "2.5.1." are the only accepted values
  ("2.5.1" is required for Promoting Interoperability Program).';
  fnCheckForSimilarErrors(█, 'INFO', 'MSH-12', 'MSH/VersionID', errorMsg, 'CONFORMANCE',
  'Field must equal value');
}

if (ELRMessage.getField('MSH/AcceptAcknowledgmentType') != null) { fnValidateFieldEqualsValue
(█, 'INFO', 'MSH-15', 'MSH/AcceptAcknowledgmentType', 'NE', '', ''); }
if (ELRMessage.getField('MSH/ApplicationAcknowledgmentType') != null) { fnValidateFieldEquals
Value(█, 'INFO', 'MSH-16', 'MSH/ApplicationAcknowledgmentType', 'NE', '', ''); }

//MSH-21 (MessageProfileIdentifier)
var hasValidMSH21 = false;
var errorMsg = 'For MSH-21.1 (MessageProfileIdentifier/EntityIdentifier), "PHLabReport-Batch"
or "PHLabReport-NoAck" are the only accepted values.';
for (var i=0; i < ELRMessage.getRepeatCount('MSH/MessageProfileIdentifier'); i++) {
  if ((ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIdentifier') == 'PHL
abReport-Batch') || (ELRMessage.getField('MSH/MessageProfileIdentifier[' + i + ']/EntityIden
tifier') == 'PHLabReport-NoAck')) {
    hasValidMSH21 = true;
  }
}
if (!hasValidMSH21) { fnCreateXMLErrorRecord (█, 'INFO', 'MSH-21.1', 'MSH/MessageProfile
Identifier[0]/EntityIdentifier', errorMsg, 'CONFORMANCE', 'Field must equal value'); }
// (END) MSH-21 (MessageProfileIdentifier) (END)

//----- MSH segment (end) -----

//----- PATIENT_RESULT group -----
for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT[' + i + ']/PATIENT/PID';
```

## Javascript

```
//----- PID segment -----
if (ELRMessage.getField(PID + '/PatientIdentifierList') != null) {
  for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
    var PID3 = PID + '/PatientIdentifierList[' + j + ']';
    if ((ELRMessage.getField(PID3 + '/IdentifierTypeCode') == 'SS') || (ELRMessage.getField(PID3 + '/AssigningAuthority/UniversalIDType') == 'SS')) {
      fnValidateFormat(█, 'INFO', 'PID-3.1', PID3 + '/IDNumber', '', '', '', 'SSN');
    }
  }
}

fnValidateDataTypeRule('TDH_Required', false, 'TDH', 'INFO', 'PID-5', PID + '/PatientName');
if (ELRMessage.getField(PID + '/PatientName') != null) {
  for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientName'); j++) {
    var PID5 = PID + '/PatientName[' + j + ']';
    fnValidateFormat(█, 'INFO', 'PID-5.1', PID5 + '/FamilyName', true, '', '', 'AlphaOnly');
    fnValidateFormat(█, 'INFO', 'PID-5.2', PID5 + '/GivenName', true, '', '', 'AlphaOnly');
    fnValidateFormat(█, 'INFO', 'PID-5.3', PID5 + '/SecondandFurtherGivenNamesorInitials', true, '', '', 'AlphaOnly');
    fnValidateFieldEqualsValue(█, 'INFO', 'PID-5.7', PID5 + '/NameTypeCode', 'L', '', '');
  }
}

//PID-7 (DateTimeOfBirth)
if (ELRMessage.getField(PID + '/DateTimeOfBirth') != null) {
  fnValidateDateTime('TS_ELR_TimeOpt', █, 'INFO', 'PID-7', PID + '/DateTimeOfBirth');

  //Date of birth cannot be before 01/01/1910 and not in the future.
  var fieldDate = ELRMessage.getField(PID + '/DateTimeOfBirth');
  if ((fieldDate != null) && (fieldDate.length >= 8) && (fieldDate.match(/\d{8}/) != null)) {
    fieldDate = new Date(Number(fieldDate.substr(0,4)),Number(fieldDate.substr(4,2))-1,Number(fieldDate.substr(6,2)));
    var pastDate = new Date(1910, 0, 1);
    var todaysDate = new Date();
    todaysDate = new Date(todaysDate.getFullYear(),todaysDate.getMonth()+1,todaysDate.getDate());
    if ((todaysDate < fieldDate) || (fieldDate < pastDate)) {
      var errorMsg = "PID-7 (DateTimeOfBirth) with value '" + ELRMessage.getField(PID + '/DateTimeOfBirth') + "' cannot be before 01/01/1910 and not in the future.";
      fnCheckForSimilarErrors(█, 'INFO', 'PID-7', PID + '/DateTimeOfBirth', errorMsg, 'CONFORMANCE', 'Invalid date');
    } // (END) if ((todaysDate < fieldDate) || (fieldDate < pastDate)) (END)
  } // (END) if ((fieldDate.length != null) && (fieldDate.length >= 8) && (fieldDate.match(/\d{8}/) != null)) (END)
} // (END) if (ELRMessage.getField(PID + '/DateTimeOfBirth') != null) (END)
// (END) PID-7 (DateTimeOfBirth) (END)

fnValidateDataTypeRule (█_Required', false, █, 'INFO', 'PID-8', PID + '/Administrative Sex');

fnValidateDataTypeRule(█_Required', false, █, 'INFO', 'PID-11', PID + '/PatientAddress');
if (ELRMessage.getField(PID + '/PatientAddress') != null) {
  for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientAddress'); j++) {
    var PID11 = PID + '/PatientAddress[' + j + ']';
    if (ELRMessage.getField(PID11 + '/City') != null) { fnValidateFormat(█, 'INFO', 'PID-11.3', PID11 + '/City', false, '', '', 'AlphaOnly'); }
    if (ELRMessage.getField(PID11 + '/ZiporPostalCode') != null) { fnValidateFormat(█, 'INFO', 'PID-11.5', PID11 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
  }
}
```

## Javascript

```

}
}

fnValidateDataTypeRule ('Required', false, 'INFO', 'PID-13', PID + '/PhoneNumberHome');
if (ELRMessage.getField(PID + '/PhoneNumberHome') != null) {
  for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PhoneNumberHome'); j++) {
    var PID13 = PID + '/PhoneNumberHome[' + j + ']';
    if (ELRMessage.getField(PID13 + '/EmailAddress') != null) { fnValidateFormat ('INFO', 'PID-13.4', PID13 + '/EmailAddress', '', '', '', 'Email'); }
    if (ELRMessage.getField(PID13 + '/AreaCityCode') != null) { fnValidateFormat ('INFO', 'PID-13.6', PID13 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(PID13 + '/LocalNumber') != null) { fnValidateFormat ('INFO', 'PID-13.7', PID13 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    if (ELRMessage.getField(PID13 + '/Extension') != null) { fnValidateFormat ('INFO', 'PID-13.8', PID13 + '/Extension', '', '', '', 'Numeric'); }
  }
}

fnValidateDataTypeRule ('Required', false, 'INFO', 'PID-22', PID + '/EthnicGroup');
if (ELRMessage.getField(PID + '/PatientDeathDateandTime') != null) { fnValidateWithinTimeFrame('INFO', 'PID-29', PID + '/PatientDeathDateandTime', 60); }

if (ELRMessage.getField(PID + '/PatientDeathIndicator') != null) { fnValidateFieldEqualsValue ('INFO', 'PID-30', PID + '/PatientDeathIndicator', 'Y_|_N', '', ''); }
//----- PID segment (end) -----

if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NK1') != null) {
  //----- NK1 segment -----
  if (ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1') > 0) {
    for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1'); j++) {
      var NK1 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']';
      fnValidateFormat ('INFO', 'NK1-1', NK1 + '/SetIDNK1', '', '', '', 'Numeric');
      if (ELRMessage.getField(NK1 + '/Name') != null) {
        for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Name'); k++) {
          var NK1_2 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Name[' + k + ']';
          fnValidateFormat ('INFO', 'NK1-2.1', NK1_2 + '/FamilyName', false, '', '', 'AlphaOnly');
          fnValidateFormat ('INFO', 'NK1-2.2', NK1_2 + '/GivenName', false, '', '', 'AlphaOnly');
        }
      }

      for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Address'); k++) {
        var NK1_4 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Address[' + k + ']';
        if (ELRMessage.getField(NK1_4 + '/City') != null) { fnValidateFormat ('INFO', 'NK1-4.3', NK1_4 + '/City', false, '', '', 'AlphaOnly'); }
        if (ELRMessage.getField(NK1_4 + '/ZiporPostalCode') != null) { fnValidateFormat ('INFO', 'NK1-4.5', NK1_4 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
      }

      for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/PhoneNumber'); k++) {
        var NK1_5 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/PhoneNumber[' + k + ']';

```

## Javascript

```

    if (ELRMessage.getField(NK1_5 + '/AreaCityCode') != null) { fnValidateFormat (
'INFO', 'NK1_5.6', NK1_5 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(NK1_5 + '/LocalNumber') != null) { fnValidateFormat (
'INFO', 'NK1_5.7', NK1_5 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
  }
}
//----- NK1 segment (end) -----
}

//----- ORDER_OBSERVATION group -----

for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION'); j++) {
  var ORC = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/ORC';
  var OBR = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBR';
  var SPM = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPM';

  //----- ORC segment -----
  fnValidateDataTypeRule ('Required', false, 'INFO', 'ORC-12', ORC + '/OrderingProvider');
  fnValidateFieldEqualsValue ('', 'INFO', 'ORC-12', ORC + '/OrderingProvider', ELRMessage
.getField(OBR + '/OrderingProvider'), 'OBR-16', OBR + '/OrderingProvider');
  if (ELRMessage.getField(ORC + '/OrderingProvider') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProvider'); k++) {
      var ORC12 = ORC + '/OrderingProvider[' + k + ']';
      if (ELRMessage.getField(ORC12 + '/IdentifierTypeCode') == 'NPI') { fnValidateFormat (
'INFO', 'ORC-12.1', ORC12 + '/IDNumber', '', 9, '123456789', 'Numeric'); }
      if (ELRMessage.getField(ORC12 + '/FamilyName') != null) { fnValidateFormat (
'INFO', 'ORC-12.2', ORC12 + '/FamilyName', false, '', '', 'AlphaOnly'); }
      if (ELRMessage.getField(ORC12 + '/GivenName') != null) { fnValidateFormat ('INFO',
'ORC-12.3', ORC12 + '/GivenName', false, '', '', 'AlphaOnly'); }
    }
  }

  if (ELRMessage.getField(ORC + '/CallBackPhoneNumber') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/CallBackPhoneNumber'); k++) {
      var ORC14 = ORC + '/CallBackPhoneNumber[' + k + ']';
      if (ELRMessage.getField(ORC14 + '/AreaCityCode') != null) { fnValidateFormat (
'INFO', 'ORC-14.6', ORC14 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
      if (ELRMessage.getField(ORC14 + '/LocalNumber') != null) { fnValidateFormat (
'INFO', 'ORC-14.7', ORC14 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
      if (ELRMessage.getField(ORC14 + '/Extension') != null) { fnValidateFormat (
'INFO', 'ORC-14.8', ORC14 + '/Extension', '', '', '', 'Numeric'); }
    }
  }

  //To determine jurisdiction for reporting: Patient address must be provided in [PID-11]
  (Patient Address).
  // However, if it is unknown and left empty, then the ordering provider must be documented.
  // If both are unknown and left empty, then the ordering facility is required.
  if ((ELRMessage.getField(PID + '/PatientAddress') == null) && ((ELRMessage.getField(ORC +
'/OrderingProvider') == null) || (ELRMessage.getField(OBR + '/OrderingProvider') == null)) &&
(ELRMessage.getField(ORC + '/OrderingFacilityName') == null)) {
    var errorMsg = 'To determine jurisdiction for reporting: Patient address must be provided
in PID-11 (Patient Address). However, if it is unknown and left empty, then the ordering pro

```

## Javascript

```

vider (ORC-12 or OBR-16) must be documented. If both are unknown and left empty, then the ordering facility (ORC-21) is required.';
    fnCreateXMLErrorRecord ( [REDACTED], 'INFO', 'PID-11', PID + '/PatientAddress', errorMsg, 'CONFORMANCE', 'Missing data');
}

fnValidateDataTypeRule ([REDACTED]Required', false, [REDACTED], 'INFO', 'ORC-21', ORC + '/OrderingFacilityName');

fnValidateDataTypeRule ([REDACTED]Required', false, [REDACTED], 'INFO', 'ORC-22', ORC + '/OrderingFacilityAddress');
if (ELRMessage.getField(ORC + '/OrderingFacilityAddress') != null) {
    fnValidateFormat([REDACTED], 'INFO', 'ORC-22.3', ORC + '/OrderingFacilityAddress/City', false, '', '', 'AlphaOnly');
    fnValidateFormat([REDACTED], 'INFO', 'ORC-22.5', ORC + '/OrderingFacilityAddress/ZiporPostalCode', '', '', '', 'ZipCode');
}

fnValidateDataTypeRule ([REDACTED]Required', false, [REDACTED], 'INFO', 'ORC-23', ORC + '/OrderingFacilityPhoneNumber');
if (ELRMessage.getField(ORC + '/OrderingFacilityPhoneNumber') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingFacilityPhoneNumber'); k++) {
        var ORC23 = ORC + '/OrderingFacilityPhoneNumber[' + k + ']';
        if (ELRMessage.getField(ORC23 + '/EmailAddress') != null) { fnValidateFormat ([REDACTED], 'INFO', 'ORC-23.4', ORC23 + '/EmailAddress', '', '', '', 'Email'); }
        if (ELRMessage.getField(ORC23 + '/AreaCityCode') != null) { fnValidateFormat ([REDACTED], 'INFO', 'ORC-23.6', ORC23 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
        if (ELRMessage.getField(ORC23 + '/LocalNumber') != null) { fnValidateFormat ([REDACTED], 'INFO', 'ORC-23.7', ORC23 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
        if (ELRMessage.getField(ORC23 + '/Extension') != null) { fnValidateFormat ([REDACTED], 'INFO', 'ORC-23.8', ORC23 + '/Extension', '', '', '', 'Numeric'); }
    }
}

if (ELRMessage.getField(ORC + '/OrderingProviderAddress') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProviderAddress'); k++) {
        var ORC24 = ORC + '/OrderingProviderAddress[' + k + ']';
        if (ELRMessage.getField(ORC24 + '/City') != null) { fnValidateFormat([REDACTED], 'INFO', 'ORC-24.3', ORC24 + '/City', false, '', '', 'AlphaOnly'); }
        if (ELRMessage.getField(ORC24 + '/ZiporPostalCode') != null) { fnValidateFormat([REDACTED], 'INFO', 'ORC-24.5', ORC24 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
    }
}

//----- ORC segment (end) -----

//----- OBR segment -----
fnValidateWithinTimeFrame([REDACTED], 'INFO', 'OBR-7', OBR + '/ObservationDateTime', 30);

if (ELRMessage.getField(OBR + '/SpecimenReceivedDateTime') != null) {
    for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/SpecimenReceivedDateTime'); k++) {
        fnValidateWithinTimeFrame([REDACTED], 'INFO', 'OBR-14', OBR + '/SpecimenReceivedDateTime[' + k + ']', 30);
    }
}

```

## Javascript

```

fnValidateDataTypeRule ('Required', false, 'INFO', 'OBR-16', OBR + '/OrderingProvider');
fnValidateFieldEqualsValue ('INFO', 'OBR-16', OBR + '/OrderingProvider', ELRMessage.getField(ORC + '/OrderingProvider'), 'ORC-12', ORC + '/OrderingProvider');
if (ELRMessage.getField(OBR + '/OrderingProvider') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderingProvider'); k++) {
    var OBR16 = OBR + '/OrderingProvider[' + k + ']';
    if (ELRMessage.getField(OBR16 + '/IdentifierTypeCode') == 'NPI') { fnValidateFormat ('INFO', 'OBR-16.1', OBR16 + '/IDNumber', '', 9, '123456789', 'Numeric'); }
    if (ELRMessage.getField(OBR16 + '/FamilyName') != null) { fnValidateFormat ('INFO', 'OBR-16.2', OBR16 + '/FamilyName', false, '', '', 'AlphaOnly'); }
    if (ELRMessage.getField(OBR16 + '/GivenName') != null) { fnValidateFormat ('INFO', 'OBR-16.3', OBR16 + '/GivenName', false, '', '', 'AlphaOnly'); }
  }
}

if (ELRMessage.getField(OBR + '/OrderCallbackPhoneNumber') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderCallbackPhoneNumber'); k++) {
    var OBR17 = OBR + '/OrderCallbackPhoneNumber[' + k + ']';
    if (ELRMessage.getField(OBR17 + '/EmailAddress') != null) { fnValidateFormat ('INFO', 'OBR-17.4', OBR17 + '/EmailAddress', '', '', '', 'Email'); }
    if (ELRMessage.getField(OBR17 + '/AreaCityCode') != null) { fnValidateFormat ('INFO', 'OBR-17.6', OBR17 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(OBR17 + '/LocalNumber') != null) { fnValidateFormat ('INFO', 'OBR-17.7', OBR17 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    if (ELRMessage.getField(OBR17 + '/Extension') != null) { fnValidateFormat ('INFO', 'OBR-17.8', OBR17 + '/Extension', '', '', '', 'Numeric'); }
  }
}

fnValidateFieldEqualsValue ('INFO', 'OBR-25', OBR + '/ResultStatus', 'F_P_C', ' ');
//----- OBR segment (end) -----

//----- OBSERVATION group -----
for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) {
  var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/OBX';

  //----- OBX segment -----
  fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-2', OBX + '/ValueType');
  fnValidateFieldEqualsValue ('INFO', 'OBX-2', OBX + '/ValueType', 'SN_NM_CE_CWE_ST_TX', ' ');

  fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-3', OBX + '/ObservationIdentifier');
  fnValidateFieldEqualsValue ('INFO', 'OBX-3.3', OBX + '/ObservationIdentifier/NameofCodingSystem', 'LN', ' ');

  fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-4', OBX + '/ObservationSubID');
  if (ELRMessage.getField(OBX + '/ObservationSubID') != null) { fnValidateFormat ('INFO', 'OBX-4', OBX + '/ObservationSubID', '', '', '', 'Numeric'); }
}

```

## Javascript

```

    if ((ELRMessage.getField(OBX + '/ValueType') == 'SN') || (ELRMessage.getField(OBX + '/Value
Type') == 'NM')) {
        fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'OBX-6', OBX + '/Units');
        fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'OBX-7', OBX + '/References
Range');
    }

    fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'OBX-8', OBX + '/Abnormal
Flags');

    fnValidateFieldEqualsValue ('█', 'INFO', 'OBX-11', OBX + '/ObservationResultStatus', 'F_|
_P_|_|_C', '', '');
    //----- OBX segment (end) -----

} // -----(END) for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) (END)
//----- OBSERVATION group (end) -----

//----- SPECIMEN group -----
for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j
+ ']/SPECIMEN'); k++) {
    var SPM = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[' + k + ']/SPM';

    //----- SPM segment -----
    fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'SPM-2', SPM + '/SpecimenID');
    fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'SPM-4', SPM + '/Specimen
Type');

    fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'SPM-17', SPM + '/SpecimenColl
ectionDateTime');
    if (ELRMessage.getField(SPM + '/SpecimenCollectionDateTime') != null) { fnValidateWithin
TimeFrame('█', 'INFO', 'SPM-17', SPM + '/SpecimenCollectionDateTime', 30); }

    if (ELRMessage.getField(SPM + '/SpecimenReceivedDateTime') != null) { fnValidateWithin
TimeFrame('█', 'INFO', 'SPM-18', SPM + '/SpecimenReceivedDateTime', 30); }
    //----- SPM segment (end) -----

    if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[' +
k + ']/OBX') != null) {
        for(var l=0; l < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' +
j + ']/SPECIMEN[' + k + ']/OBX'); l++) {
            var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/SPECIMEN[' + k + ']/
OBX[' + l + ']';

            //----- OBX segment -----
            fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'OBX-2', OBX + '/Value
Type');
            fnValidateFieldEqualsValue ('█', 'INFO', 'OBX-2', OBX + '/ValueType', 'SN_|_|_NM_|_|_CE_|_|
_CWE', '', '');

            fnValidateDataTypeRule ('█_Required', false, '█', 'INFO', 'OBX-3', OBX + '/Observation
Identifier');
            fnValidateFieldEqualsValue ('█', 'INFO', 'OBX-3.3', OBX + '/ObservationIdentifier/Nameof
CodingSystem', 'LN', '', '');

```

## Javascript

```

    fnValidateDataTypeRule ('_Required', false, ' ', 'INFO', 'OBX-4', OBX + '/Observation
SubID');
    if (ELRMessage.getField(OBX + '/ObservationSubID') != null) { fnValidateFormat (' ',
'INFO', 'OBX-4', OBX + '/ObservationSubID', '', '', '', 'Numeric'); }

    if ((ELRMessage.getField(OBX + '/ValueType') == 'SN') || (ELRMessage.getField(OBX + '/
ValueType') == 'NM')) {
        fnValidateDataTypeRule ('_Required', false, ' ', 'INFO', 'OBX-6', OBX + '/Units');
        fnValidateDataTypeRule ('_Required', false, ' ', 'INFO', 'OBX-7', OBX + '/References
Range');
    }

    fnValidateDataTypeRule ('_Required', false, ' ', 'INFO', 'OBX-8', OBX + '/Abnormal
Flags');

    fnValidateFieldEqualsValue (' ', 'INFO', 'OBX-11', OBX + '/ObservationResultStatus', 'F
_||_P_||_C', '', '');
    //----- OBX segment (end) -----

    } // -----(END) for(var l=0; l < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION[' + j + ']/SPECIMEN[' + k + ']/OBX'); l++) (END)

    } // -----(END) if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' +
j + ']/SPECIMEN[' + k + ']/OBX') != null) (END)

    } // -----(END) for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION[' + j + ']/SPECIMEN'); k++) (END)
    //----- SPECIMEN group (end) -----

    } // -----(END) for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION'); j++) (END)
    //--- ORDER_OBSERVATION group (end) ---

    } // -----(END) for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
    //--- PATIENT_RESULT group (end) ---

    //
    =====
    //_____ Conformance Rules (end)
    )
    //
    =====

```

```
next.text = dbInsertsXML;
```

## Message Collector: Input Collation

|                     |                                                                                                                                                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Collection Order    | [ECLIR - ELR 2.5.1 Message Validation/6fd292f8-d669-44e7-ba29-2cf12f762ade,ECLIR - ELR 2.5.1 Message Validation/ec50396a-2b28-4a1f-a164-4200121df8e3,ECLIR - ELR 2.5.1 Message Validation/c65755df-e855-4662-a81c-8122d88a36b9] |
| Match Property      | ELR_MESSAGE_ID                                                                                                                                                                                                                  |
| Message Set Timeout | 5                                                                                                                                                                                                                               |
| Timeout Units       | Minutes                                                                                                                                                                                                                         |

| Message Collector: Input Collation |                  |
|------------------------------------|------------------|
| Timeout Destination                | Process as Usual |

## Destinations

| Name                                                                                                                                        | Page                |
|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| Output Condition: Empty Message                                                                                                             |                     |
| Body does not contain 'errorRecord' ignoring case                                                                                           |                     |
|  ECLIR - ELR Sink.....Sink                                 | <a href="#">14</a>  |
| Error Connector                                                                                                                             | Page                |
|  db Add Field Descriptions.....Database Message Extraction | <a href="#">455</a> |



## no-op Route Error Handler In

Filter

### No-operation

On ECLIR - ELR 2.5.1 Message Validation route

#### Notes

**Purpose:** This No-operation filter receives messages from the *ECLIR - ELR 2.5.1 Message Validation Route Error Handler* when an unhandled error occurs anywhere in the route.

## Destinations

| Name                                                                                                                              | Page                |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------|
|  js Set Error properties..... JavaScript (v.1) | <a href="#">409</a> |



## Error Queue

Filter

### Error Queue

On ECLIR - ELR 2.5.1 Message Validation route

#### Configuration Properties

|               |                        |
|---------------|------------------------|
| Error Message | Sent from error filter |
|---------------|------------------------|



## db NBS\_SRTE Query Results

Filter

### Database Message Extraction

On ECLIR - ELR 2.5.1 Message Validation route

#### Notes

**Notes**

**Purpose:** This Database Lookup filter loops through the XML flat file created in the *js Create XML to query NBS\_SRTE* JavaScript filter. All results found are written to an output XML file to be processed by the *js Extract NBS\_SRTE Errors* JavaScript filter.

**Configuration Properties**

|                                  |               |                |
|----------------------------------|---------------|----------------|
| Maximum Concurrency              | 1             |                |
| Database                         | [REDACTED]    |                |
| Host                             | [REDACTED]    |                |
| Port                             | [REDACTED]    |                |
| Database Name                    | [REDACTED]    |                |
| Username                         | [REDACTED]    |                |
| Password                         | [REDACTED]    |                |
| Additional Connection Properties | Property Name | Property Value |
|                                  | [REDACTED]    | [REDACTED]     |
| Configuration File               | [REDACTED]    |                |
| Message Body Column              |               |                |
| Message Type                     | XML           |                |
| Definition                       |               |                |
| Query Timeout                    |               |                |
| Deadlock Retry Attempts          | 5             |                |
| Deadlock Retry Delay             | 100           |                |
| Socket Timeout                   | 0             |                |

**Configuration File:**

```

<statement name="Code_value_general" ide:type="executeSql" iterate="Code_value_general/query_record" ide:summary="Retrieves a record from the Code_value_general table if code is valid or if the description can be converted to a code." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Code_value_general/query_record/field_path`, '()') as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
FROM Code_value_general
WHERE code_set_nm = `/message/Code_value_general/query_record/code_set_nm`
and (code = `/message/Code_value_general/query_record/code`
or code_desc_txt = `/message/Code_value_general/query_record/code_desc_txt`
or code_short_desc_txt = `/message/Code_value_general/query_record/code_short_desc_txt`
or concept_code = `/message/Code_value_general/query_record/code`) </sql>
</statement>
<statement name="V_Race_code" ide:type="executeSql" iterate="V_Race_code/query_record" ide:summary="Retrieves a record from V_Race_code table if code is valid." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/V_Race_code/query_record/field_path`, '()') as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
FROM V_Race_code
WHERE code = `/message/V_Race_code/query_record/code`
or concept_code = `/message/V_Race_code/query_record/code` </sql>

```

## Configuration File:

```

</statement>
<statement name="Language_code " ide:type="executeSql" ide:summary="Retrieves a record from
Language_code table if code is valid." ide:allowCompoundStatements="true">
  <sql>SELECT QUOTENAME(`/message/Language_code/query_record/field_path`, '()') as field
_path, code_set_nm, code, code_desc_txt, code_short_desc_txt
FROM Language_code
WHERE code_set_nm = `/message/Language_code/query_record/code_set_nm`
and code = `/message/Language_code/query_record/code`</sql>
</statement>
<statement name="V_State_county_code_value" ide:type="executeSql" iterate="V_State_county
_code_value/query_record" ide:summary="Returns a record from V_State_county_code_value table
if code or description is valid." ide:allowCompoundStatements="true" user:iterationRootMessage
Path="message">
  <sql>SELECT QUOTENAME(`/message/V_State_county_code_value/query_record/field_path`, '()')
as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
FROM V_State_county_code_value
WHERE code_set_nm = `/message/V_State_county_code_value/query_record/code_set_nm`
and (code = `/message/V_State_county_code_value/query_record/code`
or code_desc_txt = `/message/V_State_county_code_value/query_record/code_desc_txt`
or code_short_desc_txt = `/message/V_State_county_code_value/query_record/code_short_desc
txt`
or concept_code = `/message/V_State_county_code_value/query_record/code`)</sql>
</statement>
<statement name="State_code" ide:type="executeSql" iterate="State_code/query_record" ide:sum
mary="Validate state code from code or text." ide:allowCompoundStatements="true" user:iteration
RootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/State_code/query_record/field_path`, '()') as field_path,
Code_value_general.code_set_nm, Code_value_general.code, Code_value_general.code_desc_txt,
Code_value_general.code_short_desc_txt, Code_value_general.concept_code, State_code.state_nm
FROM Code_value_general
FULL OUTER JOIN State_code
ON Code_value_general.code = State_code.state_cd
WHERE Code_value_general.code_set_nm = 'STATE_CCD'
and (Code_value_general.code = `/message/State_code/query_record/code`
or State_code.state_nm = `/message/State_code/query_record/code`
or Code_value_general.concept_code = `/message/State_code/query_record/code`)</sql>
</statement>
<statement name="Labtest_loinc" ide:type="executeSql" iterate="Labtest_loinc/query_record"
ide:summary="Returns a record from Labtest_loinc table if a local code can be translated to a
LOINC." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record/field_path`, '()') as field
_path, lab_test_cd, loinc_cd AS LocalToLOINC
FROM Labtest_loinc
WHERE status_cd = 'A' AND laboratory_id = `/message/Labtest_loinc/query_record/lab_id`
AND lab_test_cd = `/message/Labtest_loinc/query_record/lab_test_cd`</sql>
  <statement name="Check LOINC condition" ide:type="executeSql" test="NotNull(@LocalToLOINC
)" ide:summary="If the local code has an equivalent LOINC, check table LOINC_condition to val
idate." ide:allowCompoundStatements="true">
    <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record[1]/field_path`, '()') as field
_path, loinc_cd
FROM LOINC_condition
WHERE loinc_cd = @LocalToLOINC</sql>
  </statement>
  <statement name="Check LOINC_code" ide:type="executeSql" test="NotNull(@LocalToLOINC)" ide
:summary="If the local code has an equivalent LOINC, check table LOINC_code to validate." ide
:allowCompoundStatements="true">
    <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record[1]/field_path`, '()') as field
_path, loinc_cd

```

## Configuration File:

```

FROM LOINC_code
WHERE loinc_cd = @LocalToLOINC</sql>
</statement>
</statement>
<statement name="LOINC_condition" ide:type="executeSql" iterate="LOINC_condition/query_record" ide:summary="Returns the LOINC code if it is found in LOINC_condition table." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/LOINC_condition/query_record/field_path`, '()') as field_path, loinc_cd
  FROM LOINC_condition
  WHERE loinc_cd = `/message/LOINC_condition/query_record/loinc_cd`</sql>
</statement>
<statement name="LOINC_code" ide:type="executeSql" iterate="LOINC_code/query_record" ide:summary="Returns a record from LOINC_code table if LOINC code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/LOINC_code/query_record/field_path`, '()') as field_path, loinc_cd
  FROM LOINC_code
  WHERE loinc_cd = `/message/LOINC_code/query_record/loinc_cd`</sql>
</statement>
<statement name="Snomed_condition" ide:type="executeSql" iterate="Snomed_condition/query_record" ide:summary="Returns a record from Snomed_condition table if SNOMED code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Snomed_condition/query_record/field_path`, '()') as field_path, snomed_cd
  FROM Snomed_condition
  WHERE snomed_cd = `/message/Snomed_condition/query_record/snomed_cd`</sql>
</statement>
<statement name="Snomed_code " ide:type="executeSql" iterate="Snomed_code/query_record" ide:summary="Returns a record from Snomed_code table if SNOMED code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Snomed_code/query_record/field_path`, '()') as field_path, snomed_cd
  FROM Snomed_code
  WHERE snomed_cd = `/message/Snomed_code/query_record/snomed_cd`</sql>
</statement>
<statement name="dummy_record" ide:type="executeSql" ide:summary="This creates a dummy record in the output XML in the event none of the queries return a result." ide:allowCompoundStatements="true">
  <sql>SELECT SYSDATETIME() AS 'dummy_time_record'</sql>
</statement>
</config>

```

## Destinations

| Name                                             | Page |
|--------------------------------------------------|------|
| js Extract NBS_SRTE Errors..... JavaScript (v.1) | 467  |



## no-op ELR Msgs In

## No-operation

On ECLIR - ELR 2.5.1 Message Validation route

Filter

## Notes